

Special Participation B: Gemini Pro-3 HW10

Shervin Goudarzi

December 1, 2025

Model Performance Summary

Question Name	Model Performance
PytorchTransformer	One-Shot
Self-Attention (Content)	One-Shot
Self-Attention (Position)	One-Shot
scaled_dot_product_attention	One-Shot
MultiheadAttention	One-Shot
TransformerLayer	One-Shot
make_positions	One-Shot

Initial Setup

User: You are assisting an undergraduate in their deep learning class. Only complete the sections that says "TODO" in the code for the notebook. Don't output anything now but simply output the desired code within the TODO with clear explanation of how you reached that decision when i give you desired question.

Gemini: I have analyzed the uploaded notebook. I see the sections marked for implementation:

- PytorchTransformer Class
- Self-Attention by Content
- Self-Attention by Position
- Self-Attention by Content and Position

I am ready to provide the code and clear explanations for any of these sections. Which part would you like to work on first?

Part 1: Hand-Designed Transformer (q_hand_transformer.ipynb)

User: Give the code for the PytorchTransformer class and the corresponding forward pass.

Gemini: Here is the implementation for the PytorchTransformer class.

Explanation of the Implementation

1. `__init__`: We define Q_m , K_m , and V_m using `nn.Linear` with `bias=False`. The input dimension is adjusted for positional encodings if necessary.
2. `forward`:
 - Handles positional encoding concatenation.
 - Projects input to Q, K, V.
 - Calculates Scaled Dot-Product Attention: $\text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$.

```

1 class PytorchTransformer(nn.Module):
2     def __init__(self, input_dim, qk_dim, v_dim, pos_dim=None, max_seq_len=10):
3         super().__init__()
4         if pos_dim is not None:
5             self.pos = nn.Embedding(max_seq_len, pos_dim)
6         else:
7             self.pos = None
8         in_dim = input_dim
9         if pos_dim is not None:
10             in_dim += pos_dim
11
12         # Define projection layers
13         self.Qm = nn.Linear(in_dim, qk_dim, bias=False)
14         self.Km = nn.Linear(in_dim, qk_dim, bias=False)
15         self.Vm = nn.Linear(in_dim, v_dim, bias=False)
16
17         self.d_k = qk_dim
18
19     def forward(self, seq):
20         # 1. Handle Positional Encodings
21         if self.pos is not None:
22             seq_len = seq.shape[0]
23             positions = torch.arange(seq_len, device=seq.device)
24             pos_embeddings = self.pos(positions)
25             seq = torch.cat([seq, pos_embeddings], dim=1)
26
27         # 2. Compute Q, K, V
28         Q = self.Qm(seq)
29         K = self.Km(seq)
30         V = self.Vm(seq)
31
32         # 3. Scaled Dot-Product Attention
33         scores = torch.matmul(Q, K.t()) / math.sqrt(self.d_k)
34         attn_weights = torch.softmax(scores, dim=-1)
35         out = torch.matmul(attn_weights, V)
36
37         return out

```

User: For the forward function, this is what the solution said: [User provided solution code]. Is your code correct/matching the solution?

Gemini: Yes, my code is functionally identical to the solution. There are two minor syntax differences:

1. **Positional Encodings:** The solution slices weights directly (`self.pos.weight[:len]`), while I used the layer call. Both return the same values.
2. **Transpose:** The solution uses `.transpose(-2, -1)`, I used `.t()`. Since inputs are 2D here, they are identical.

Here is the updated code matching the solution's syntax for consistency:

```

1     def forward(self, seq):
2         if self.pos is not None:
3             seq = torch.cat([seq, self.pos.weight[:seq.shape[0]]], dim=-1)
4
5         Q = self.Qm(seq)
6         K = self.Km(seq)
7         V = self.Vm(seq)
8
9         dot = (Q @ K.transpose(-2, -1)) / math.sqrt(self.d_k)
10        softmax_dot = torch.softmax(dot, dim=-1)
11        out = softmax_dot @ V
12
13        return out

```

User: Ok great, now solve the self-attention by content question.

Gemini: To solve "Self-Attention: Attention by Content", we need the transformer to perform the identity operation.

Logic:

- **Values (Vm):** Identity matrix to preserve content.

- **Keys/Queries (Km, Qm):** Since inputs are orthogonal one-hot vectors, Identity matrices work. However, we scale **Qm** by 10 to make the dot products large. This ensures the softmax is "sharp" (close to 1 for the target, 0 for others), avoiding blurry averages.

```

1 # Key Matrix: Identity matrix
2 Km = np.eye(3)
3
4 # Query Matrix: Scaled Identity matrix
5 # Scale by 10 to ensure sharp Softmax
6 Qm = np.eye(3) * 10
7
8 # Value Matrix: Identity matrix
9 Vm = np.eye(3)

```

User: Awesome now solve the TODO here: Self-Attention by Position.

Gemini: We need to copy the first token (at Position 0) to all outputs.

Logic:

1. **Positional Encoding:** Identity matrix size 4 (one-hot for positions).
2. **Key (Km):** Needs to activate only for Position 0. Position 0 is index 3 in the concatenated vector. Set $Km[3, 0] = 10$.
3. **Query (Qm):** Needs to match the Key at Pos 0 always. Set weights for all position indices (3 to 6) to 1.
4. **Value (Vm):** Identity for the content part (indices 0-2) to copy the token.

```

1 # 1. Positional Encodings (One-hot)
2 pos = np.eye(4)
3
4 # 2. Key Matrix (High value only for Pos 0 / index 3)
5 Km = np.zeros((7, 1))
6 Km[3, 0] = 10
7
8 # 3. Query Matrix (Active for all position indices)
9 Qm = np.zeros((7, 1))
10 Qm[3:, 0] = 1
11
12 # 4. Value Matrix (Pass content through)
13 Vm = np.zeros((7, 3))
14 Vm[:3, :] = np.eye(3)

```

User: Explain this [Content (3) | Position (4)] please and why the index at index 3 corresponds to position 0.

Gemini: This notation represents the concatenated vector structure.

- **Content (Indices 0-2):** The token vector (length 3).
- **Position (Indices 3-6):** The position vector (length 4).

Since `pos` is an Identity matrix, Position 0 is $[1, 0, 0, 0]$. When concatenated:

$$\underbrace{[c_0, c_1, c_2]}_{\text{Content}}, \underbrace{[1, 0, 0, 0]}_{\text{Position 0}}$$

The **1** for Position 0 lands exactly at **Index 3**. That is why `Km` looks at Index 3 to detect Position 0.

Part 2: Transformer Summarization (q_summarize_part_1.ipynb)

User: ok now here is the new notebook check for all the TODOS similar to above, dont solve any problem yet. [User uploaded file].

Gemini: I analyzed the notebook. The tasks are:

1. Implement scaled_dot_product_attention.
2. Implement MultiheadAttention.
3. Implement TransformerLayer.
4. Implement Transformer.make_positions.

User: Ok solve scaled_dot_product_attention.

Gemini: Here is the implementation. It handles score calculation, causal masking (using future_mask), and padding masking.

```
1 def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False):
2     d_head = q.size(-1)
3
4     # 1. Raw scores: Q * K^T / sqrt(d)
5     # k transpose: [n, d_head, src_len]
6     scores = torch.bmm(q, k.transpose(1, 2)) / (d_head ** 0.5)
7
8     # 2. Causal Masking
9     if causal:
10         tgt_len, src_len = q.size(1), k.size(1)
11         mask = future_mask[:tgt_len, :src_len].to(q.device)
12         scores = scores + mask
13
14     # 3. Padding Masking
15     if key_padding_mask is not None:
16         # Mask where value is 1 (pad)
17         scores = scores.masked_fill(key_padding_mask.unsqueeze(1) == 1, float("-inf"))
18
19     # 4. Softmax and Output
20     attn_weights = F.softmax(scores, dim=-1)
21     output = torch.bmm(attn_weights, v)
22
23     return output
```

User: Nice! Now solve, MultiheadAttention forward pass.

Gemini: This involves linear projections, splitting the heads (reshaping and transposing), flattening batch/heads for attention, and recombining.

```
1 def forward(self, q, k, v, key_padding_mask=None, causal=False):
2     bsz, tgt_len, _ = q.size()
3     src_len = k.size(1)
4
5     # 1. Projections
6     q = self.q_proj(q)
7     k = self.k_proj(k)
8     v = self.v_proj(v)
9
10    # 2. Split Heads & Flatten
11    q = q.view(bsz, tgt_len, self.n_heads, self.d_head).transpose(1, 2)
12    k = k.view(bsz, src_len, self.n_heads, self.d_head).transpose(1, 2)
13    v = v.view(bsz, src_len, self.n_heads, self.d_head).transpose(1, 2)
14
15    q = q.reshape(bsz * self.n_heads, tgt_len, self.d_head)
16    k = k.reshape(bsz * self.n_heads, src_len, self.d_head)
17    v = v.reshape(bsz * self.n_heads, src_len, self.d_head)
18
19    # 3. Expand Mask for Heads
20    if key_padding_mask is not None:
21        key_padding_mask = key_padding_mask.view(bsz, 1, src_len)\
22            .expand(-1, self.n_heads, -1).reshape(bsz * self.n_heads, src_len)
23
```

```

24     # 4. Attention
25     attn_out = scaled_dot_product_attention(q, k, v, key_padding_mask, causal)
26
27     # 5. Recombine Heads
28     attn_out = attn_out.view(bsz, self.n_heads, tgt_len, self.d_head)
29     attn_out = attn_out.transpose(1, 2)
30     attn_out = attn_out.reshape(bsz, tgt_len, self.d_head * self.n_heads)
31
32     # 6. Output Projection
33     output = self.o_proj(attn_out)
34     return output

```

User: Great, now solve the TransformerLayer forward pass.

Gemini: Implementation of the standard Transformer layer structure: Self-Attention → Residual/Norm → Cross-Attention (if Decoder) → Residual/Norm → FFN → Residual/Norm.

```

1  def forward(self, x, padding_mask, encoder_out=None, encoder_padding_mask=None):
2      # 1. Self-Attention
3      residual = x
4      out = self.self_attn(x, x, x, key_padding_mask=padding_mask, causal=self.
is_decoder)
5      out = self.self_attn_drop(out)
6      x = self.self_attn_ln(x + out)
7
8      # 2. Cross-Attention (Decoder Only)
9      if self.is_decoder:
10         if encoder_out is None: raise ValueError("encoder_out required for decoder")
11         residual = x
12         out = self.cross_attn(x, encoder_out, encoder_out,
13                                key_padding_mask=encoder_padding_mask, causal=False)
14         out = self.cross_attn_drop(out)
15         x = self.cross_attn_ln(x + out)
16
17     # 3. FFN
18     residual = x
19     out = self.fc2(F.relu(self.fc1(x)))
20     out = self.ffn_drop(out)
21     x = self.ffn_ln(x + out)
22
23     return x

```

User: Now finally implement the make_positions of the Transformer class.

Gemini: This generates a sequence of indices $[0, 1, \dots, \text{len} - 1]$ and expands it to the batch size.

```

1  def make_positions(self, input_ids, padding_mask):
2      bsz, seq_len = input_ids.shape
3      # Generate range [0, 1, ..., seq_len - 1]
4      positions = torch.arange(seq_len, device=input_ids.device)
5      # Expand to batch size
6      return positions.unsqueeze(0).expand(bsz, -1)

```